

Graph Paths I

Consider a directed graph that has n nodes and m edges. Your task is to count the number of paths from node 1 to node n with exactly k edges.

Input

The first input line contains three integers n , m and k : the number of nodes and edges, and the length of the path. The nodes are numbered $1, 2, \dots, n$.

Then, there are m lines describing the edges. Each line contains two integers a and b : there is an edge from node a to node b .

Output

Print the number of paths modulo 10^9+7 .

Constraints

$1 \leq n \leq 100$
 $1 \leq m \leq n(n-1)$
 $1 \leq k \leq 10^9$
 $1 \leq a, b \leq n$

Example

Input:
3 4 8
1 2
2 3
3 1
3 2

Output:
2

Explanation:

1 -> 2 -> 3 -> 1 -> 2 -> 3 -> 1 -> 2 -> 3
1 -> 2 -> 3 -> 2 -> 3 -> 2 -> 3 -> 2 -> 3

Why normal graph DP fails

Let's think in the most straightforward DP way.

Define:

$dp[len][v]$ = number of ways to reach node v using exactly len edges

Transition:

$dp[len+1][to] += dp[len][from]$ (if edge from $\rightarrow$ to exists)

This is perfectly valid.

But:

- $len = k$ can be 10^9
- You cannot simulate step by step

So we need:

- To "jump" many steps at once
- Just like Fibonacci
- Using matrix exponentiation

Ask the key DP question:

"What is my state at a given step?"

Answer:

For every node, how many ways I can be at that node

So the state is not a single number, but a vector of size n .

Define the DP state

Let:

$$S(t) = \begin{bmatrix} \text{ways to be at node 1 after } t \text{ edges} \\ \text{ways to be at node 2 after } t \text{ edges} \\ \vdots \\ \text{ways to be at node } n \text{ after } t \text{ edges} \end{bmatrix}$$

This is exactly like:

`vector<long long> dp(n);`

How does the state change after one edge?

Suppose we are at step t .

For a fixed node v :

You can reach v from any node u that has an edge $u \rightarrow v$

So:

$$S(t+1)[v] = \sum_{u \rightarrow v} S(t)[u]$$

This is linear, no conditionals, no branching.

Convert this transition into a matrix

We want:

$$S(t+1) = M \cdot S(t)$$

Where:

- M is an $n \times n$ matrix
- $M[v][u]$ tells how much node u contributes to node v

From the formula:

$$M[v][u] = \begin{cases} \text{number of edges from } u \rightarrow v, & \text{if an edge exists,} \\ 0, & \text{otherwise.} \end{cases}$$

This matrix is simply the adjacency matrix of the graph.

Important mental note:

Matrix exponentiation on graphs = powering the adjacency matrix.

Define the initial state

At the beginning (0 edges):

- You are at node 1
- Nowhere else

$$S(0) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Code

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1000000007;

struct Matrix {
    vector<vector<ll>>> a;
    int n;
    Matrix(int n) : n(n) {
        a.assign(n, vector<ll>(n, 0));
    }
};

// Multiply two matrices
Matrix multiply(const Matrix &A, const Matrix &B) {
    int n = A.n;
    Matrix C(n);

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            if (A.a[i][k] == 0) continue;
            for (int j = 0; j < n; j++) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

// Fast matrix exponentiation
Matrix matrix_power(Matrix base, long long exp) {
    int n = base.n;
    Matrix result(n);

    // Identity matrix
    for (int i = 0; i < n; i++)
        result.a[i][i] = 1;

    while (exp > 0) {
        if (exp & 1)
            result = multiply(result, base);
        base = multiply(base, base);
        exp >>= 1;
    }
    return result;
}

int main() {
    int n, m;
    long long k;
    cin >> n >> m >> k;

    Matrix M(n);

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        u--; v--;
        M.a[u][v] = (M.a[u][v] + 1) % MOD;
    }

    Matrix Mk = matrix_power(M, k);

    // Initial state: S(0) = [1, 0, 0, ...]
    // We want S(k)[n-1]
    ll answer = Mk.a[0][n - 1];
    cout << answer << "\n";

    return 0;
}
```